

Document Number: P3174R0

Date: 2024-03-09

Audience: SG16

Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2023-10-11 through 2024-02-21

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [October 11th, 2023](#)
- [October 25th, 2023](#)
- [November 29th, 2023](#)
- [January 10th, 2024](#)
- [January 24th, 2024](#)
- [February 7th, 2024](#)
- [February 21st, 2024](#)

Previously published SG16 meeting summary papers:

- [P1080: SG16: Unicode meeting summaries 2018/03/28 - 2018/04/25](#)
- [P1137: SG16: Unicode meeting summaries 2018/05/16 - 2018/06/20](#)
- [P1237: SG16: Unicode meeting summaries 2018/07/11 - 2018/10/03](#)
- [P1422: SG16: Unicode meeting summaries 2018/10/17 - 2019/01/09](#)
- [P1666: SG16: Unicode meeting summaries 2019/01/23 - 2019/05/22](#)
- [P1896: SG16: Unicode meeting summaries 2019/06/12 - 2019/09/25](#)
- [P2009: SG16: Unicode meeting summaries 2019-10-09 through 2019-12-11](#)
- [P2179: SG16: Unicode meeting summaries 2020-01-08 through 2020-05-27](#)
- [P2217: SG16: Unicode meeting summaries 2020-06-10 through 2020-08-26](#)
- [P2253: SG16: Unicode meeting summaries 2020-09-09 through 2020-11-11](#)
- [P2352: SG16: Unicode meeting summaries 2020-12-09 through 2021-03-24](#)
- [P2397: SG16: Unicode meeting summaries 2021-04-14 through 2021-05-26](#)
- [P2512: SG16: Unicode meeting summaries 2021-06-09 through 2021-12-15](#)
- [P2605: SG16: Unicode meeting summaries 2022-01-12 through 2022-06-08](#)
- [P2678: SG16: Unicode meeting summaries 2022-06-22 through 2022-09-28](#)
- [P2766: SG16: Unicode meeting summaries 2022-10-12 through 2022-12-14](#)
- [P2891: SG16: Unicode meeting summaries 2023-01-11 through 2023-05-10](#)
- [P2995: SG16: Unicode meeting summaries 2023-05-24 through 2023-09-27](#)

October 11th, 2023

Draft agenda:

- [P1729R3: Text Parsing](#):
 - Continue review.

Attendees:

- Corentin Jabot
- Elias Kosunen
- Hubert Tong
- Nathan Owen
- Robin Leroy
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- [P1729R3: Text Parsing](#):
 - *[Editor's note: D1729R3 was the active paper under discussion at the telecon. The agenda and links used here reference P1729R3 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
 - Elias presented the changes in the draft P1729R3:
 - `std::scan` now returns a subrange for the unparsed input rather than just an iterator to the start of the range.
 - As noted in the revision history, changes requested during the last SG16 review with respect to whitespace, locale, and encoding concerns have been made.
 - Victor asked if returning a subrange will be less efficient since it requires passing an iterator pair or an iterator and size pair.
 - Elias responded that the overhead is expected to be negligible relative to the convenience provided by returning the sentinel.
 - Elias commented that, per section 3.6, "Scanning an user-defined type", the second template parameter for `std::scanner` now has `char` as a default argument.
 - Elias reviewed the changes in section 4.2, "Format strings" to define whitespace in terms of the `Unicode Pattern_White_Space` property.
 - Victor asked why LEFT-TO-RIGHT MARK and RIGHT-TO-LEFT MARK are considered whitespace.

- Robin responded that these code points can be used to prevent directionality properties from one token from affecting how the characters of an adjacent token are displayed.
- Tom asked for confirmation that there is no desire or need for scanning to consider bidirectional concerns; e.g., scanning should always follow memory order, not logical order.
- Robin referenced the examples in [section 1.3.2, "Usability issues arising from bidirectional reordering"](#) of [UTS #55, "Unicode Source Code Handling"](#) that demonstrate how the Unicode Bidirectional Algorithm can produce unreadable text.
- Victor requested the addition of some bidirectional examples and asked Robin if he could offer some suggestions that would be relevant for scanning.
- Robin responded in chat to see the examples in [section 4.1.1, "Bidirectional Ordering"](#) of [UAX #31, "Unicode Identifiers and Syntax"](#).
- Elias agreed that examples can be added.
- Tom noted that, when the input is not known to be in a UTF encoding, that the set of whitespace characters will need to be implementation-defined.
- Elias agreed and stated those details will be added later.
- Elias directed attention to section 4.3.5.1, "Design discussion: Thousands separator grouping checking" and noted that iostreams enforces grouping separators.
- Tom asked for confirmation that iostreams only enforces that, if grouping separators are present, that they are in the expected locations and that they aren't required to be present.
- Elias confirmed.
- Victor asserted that `std::scan` should do what iostreams does and stated that programmers that want different behavior can implement that themselves.
- Elias suggested the behavior could potentially be changed later if desired.
- Victor replied that it is generally more difficult to introduce an error where one was not previously reported than it is to relax an error that was previously reported.
- Elias noted that some `scanf()` implementations have an extension that allows ' to be recognized as a grouping separator.
- Tom asked if that separator is handled like it is in C++ where it can appear anywhere any number of times.
- Elias responded that it is recognized as an alternate grouping separator, so no.
- Victor explained that [{fmt}](#) briefly supported that feature but that it was removed.
- Victor opined that support for that feature probably isn't needed.
- Elias acknowledged that support for it could always be added later.
- Corentin agreed with Victor, expressed a desire to eventually replace locale support with something based on ICU someday, and encouraged avoidance of innovation with locale features.
- Elias stated that he would not proceed further with the alternate separator.

- Elias pointed out that section 4.5, "Argument passing, and return type of scan", now specifies that `std::scan` returns a subrange.
- Elias observed a markup error in the last paragraph of that section; ">" appears where ">" was intended to encode ">".
- Elias claimed that the return of a subrange consisting of an iterator and sentinel pair is novel and is done because the sentinel is always available but converting it to an iterator would require more work to advance an iterator to the sentinel position.
- Tom encouraged Elias to contact the SG9 chair to arrange a discussion.
- Elias proclaimed that a better name is needed for the proposed `borrowed_ssubrange_t` and explained that the extra "s" stands for sentinel.
- Steve agreed and stated that, as is, that name looks like a typo.
- Steve recommended spelling the name out since this isn't one that programmers would have to write often anyway.
- Corentin suggested that it might be possible to change `borrowed_subrange` to support an iterator and sentinel subrange.
- Elias replied that doing so might impact ABI.
- Corentin recommended discussing it in SG9.
- Elias presented section 4.6, "Error handling", and the recently added `value_out_of_range` enumerator added to `scan_error::code_type`.
- Elias explained that the `strtol()` family of interfaces allow a programmer to differentiate between overflow and underflow using a combination of the return value and `errno`, but that `std::scan` as proposed would not be able to support that.
- Victor reported having previously needed to be able to differentiate between underflow and overflow.
- Tom stated that it sounds like there is some motivation for more granular errors.
- Corentin argued that isn't a question for SG16 to answer.
- Elias reported that there are a lot of potential error conditions and argued that adding a different error code for each is probably undesirable.
- Corentin asked if a distinct error code is needed for encoding errors.
- Elias responded that there had been discussion about that during the previous review and that we'll get to that section shortly.
- Corentin asserted that it would be useful to provide an iterator or index to the position within the input where an error occurred.
- Victor agreed.
- Victor suggested it would make sense to provide more granular error handling for builtin types.
- Victor requested some additional examples and noted that there are unique error cases for floating-point types.
- Elias mentioned that an example has been added to section 4.10, "Locales".
- Elias stated that section 4.11, "Encoding" was added for the R3 revision.
- Elias summarized discussion from the last SG16 review; that ill-formed code unit sequences be handled similar to floating-point NaN values in that they don't match

anything.

- Victor suggested that "invalidly encoded code points" should be changed to something like "ill-formed code unit sequences".
- Corentin asked if the intent is to supply replacement characters for ill-formed code unit sequences.
- Elias replied negatively and explained that the intent is to allow use of `std::string_view` as a result type that refers to matched characters in the input; that support precludes substitution of replacement characters.
- Elias stated that these sequences are instead handled like non-characters.
- Elias acknowledged that this design means that unsanitized input won't be validated and that ill-formed code unit sequences may persist in the output.
- Corentin noted the implication; that values returned by `std::scan` can't be trusted and lack of verification can result in UB and security issues.
- Elias agreed that there is a security aspect since the input could be arbitrary user provided input.
- Victor opined that the proposed behavior seems reasonable and consistent with other scan-like functions.
- Victor suggested updating the paper to compare the proposed behavior with `scanf()`.
- Steve noted that, even if the input was mutable, rewriting replacement characters into the buffer is not an option since the space needed for the encoded replacement character might require a longer buffer.
- Steve explained that Zach's proposed transcoding facilities could be used to pipe input that has not been validated for encoding concerns into the scanner such that replacement characters are proactively substituted.
- *[Editor's note: The input produced by such a pipeline would not provide a contiguous range of elements and would presumably not be usable with a `std::string_view` result type.]*
- Steve expressed a preference for features that compose.
- Victor asserted that it should be possible to use `std::scan` with binary data and that ill-formed code unit sequences should therefore not be unconditionally rejected.
- Corentin agreed that support for binary data is an important concern and referred to a comment [Tom made in a message to the SG16 mailing list](#) about the potential use of a `{:?}` format specifier for byte precise scanning.
- Corentin expressed uncertainty regarding how important it is to handle mixed binary and text.
- Corentin noted that the proposed design provides different guarantees for different types; result objects of `int` and `float` type will always hold valid values, but a string type might hold garbage.
- Corentin worried that programmers might expect a validly encoded string and be surprised.

- Victor claimed that it is not possible to determine what is and is not garbage since programmers do use string types like `std::string_view` with binary data.
- Victor asserted that we should not try to guess the programmer's intent.
- Tom agreed that we should not assume the programmer's intent and observed that providing a facility to allow them to express their intent could be ok.
- Elias reported that the example that Tom included in the [agenda announcement](#) has been added as example 6 in section 4.3.8, "Type specifiers: CharT".
- *[Editor's note: the example involves a scan of the first code unit of a multiple code unit sequence followed by a scan of a string that then interprets the remainder of the code unit sequence as an ill-formed sequence.]*
- Corentin noted that scanning strings requires recognizing spaces and asked if there is a use case for a space separated sequence of random bytes.
- Corentin surmised that, if that use case is important, then it should influence the design.
- Victor recognized Corentin's observation regarding spaces and random bytes as important.
- Victor stated that the behavior described for the example in the paper matches his expectations.
- Elias argued that the entire input should not be sanitized due to processing overhead.
- Elias affirmed that an invalidly encoded string could be handled as an error.
- Tom asserted it would be useful to allow the programmer to express their intent with a type specifier.
- Tom noted that the ability to do so would allow for the kinds of encoding guarantees that programmers might expect and argued that this should be the default behavior.
- Elias agreed that would be useful.
- Elias stated that he will have to evaluate further how that fits into the design but that it sounds manageable.
- Tom asked if `signed char` and `unsigned char` are handled as character or integer types.
- Elias responded that they are treated as integer types.
- Tom noted that is consistent with `std::format()`.
- Elias added that it is also consistent with `iostream`.
- Victor conveyed a lack of enthusiasm for an additional format specifier due to the increased complexity.
- Tom suggested relying on the type system instead; perhaps `std::span<char>` could be used to scan a "binary string".
- Victor agreed and suggested there could be another type to represent a broken code unit.
- Corentin nominated `std::byte`.
- Tom noted that `std::byte` wouldn't work for wide strings.
- Corentin countered that wide strings aren't used for binary data.

- Tom responded that a programmer might want to be able to read a lone surrogate.
- Victor reported that `std::format()` formats `std::byte` as an unsigned integer.
- Tom summarized his impression of the consensus at this point; the design is good, but some progress is needed regarding handling of text vs binary input.
- Corentin expressed a penchant for the design in general.
- Elias requested that the meeting minutes be published before October 15th so that they would be available for reference by the R3 paper in time for the next mailing deadline.
- Tom said he would try.
- *[Editor's note: Tom provided a rough draft of the minutes prior to the 15th and that sufficed for Elias' purposes.]*
- Tom announced that the next meeting will be held 1023-10-25 and that there are some LWG issues to be discussed, including ones involving everyone's favorite locale facet, `std::codecvt`.
- Hubert stated that he might soon have a paper that discusses use of `$` in identifiers.

October 25th, 2023

Draft agenda:

- `charN_t`, `char_traits`, `codecvt`, and `iostreams`:
 - [P2873R0: Remove Deprecated Locale Category Facets For Unicode from C++26](#).
 - [LWG 3767: `codecvt<charN\_t, char8\_t, mbstate\_t>` incorrectly added to locale](#).
 - [LWG 2959: `char\_traits<char16\_t>::eof` is a valid UTF-16 code unit](#).
 - [SG16 #32: `std::char\_traits<char16\_t>::eof\(\)` requires `uint\_least16\_t` to be larger than 16 bits](#).
 - [SG16 #33: A correct `codecvt` facet that works with basic `filebuf` can't do UTF conversions](#).

Attendees:

- Alisdair Meredith
- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark de Wever
- Nathan Owens
- Peter Brett
- Robin Leroy
- Steve Downey
- Tom Honermann

- Victor Zverovich

Meeting summary:

- PBrett announced that he will be retiring from C++ standardization efforts for the foreseeable future starting in November.
- Several people voiced disappointment and wished Peter well.
- `charN_t`, `char_traits`, `codecvt`, and `iostreams`:
 - Tom reported having reached out to the WG21 ABI review group to ask if there were any known ABI tricks that implementors might deploy if [LWG 2959](#) ([char_traits<char16_t>::eof is a valid UTF-16 code unit](#)) were to be fixed in the obvious way; by mapping the `int_type` member alias to a larger type.
 - Tom summarized their response; no tricks were identified; suggestions included defining a replacement type for the `std::char_traits<char16_t, char, std::mb_state>` specialization that could be explicitly used in its place.
 - Corentin replied that a replacement type doesn't solve the user problem.
 - Corentin reported intent to submit a proposal to deprecate user specializations of `std::char_traits`.
 - Corentin asked if Tom had asked the libcpp maintainers directly regarding their thoughts on the issue.
 - Tom reported that he has not.
 - Corentin suggested that doing so might be helpful.
 - Tom reported having audited uses of the `int_type` and related members of `std::char_traits` throughout the standard and having found that they are only used within `iostreams` and, since the standard only requires `iostreams` to support `char` and `wchar_t`, changing `int_type` for the `char16_t` specialization appears to be a viable option.
 - *[Editor's note: Tom's audit rediscovered information that was already known and had been reported in [a comment on SG16 issue #32](#) back in 2018.]*
 - Hubert stated that the libcpp implementation of `iostreams` uses the `eof()` member of `std::char_traits` as a sentinel value to determine if a fill character has been specified via the `std::setfill()` I/O manipulator.
 - *[Editor's note: The libcpp implementation of `std::basic_ios` has a private data member named `__fill_` of type `int_type` that is initialized to `eof()`. When a fill character is needed, a comparison is performed against `eof()` to determine if a fill character has been set or whether the (possibly widened) default fill character should be used.]*
 - Hubert noted this as an issue for the `wchar_t` `iostream` and `std::char_traits` specializations.
 - Tom noted that, for `wchar_t` the EOF value is specified by `WEOF` and asked if it is known to have a value other than `-1` anywhere.
 - Hubert responded that he was not aware of other values being used, but that the value is problematic because programmers can use that value.

- *[Editor's note: Microsoft's `wchar.h` header defines `WEOF` as `((wint_t)(0xFFFF))` which is equivalent to `-1` converted to `wint_t` (unsigned short).]*
- Tom acknowledged the concern as applicable to the `wchar_t` specialization and that it can be treated as a separable issue.
- Corentin reported that the C++ standard appears to be missing a definition for `WEOF`.
- Jens responded that the C++ standard has an exposition value of `"*see below"` that is intended to redirect to the C library.
- Jens noted the redirection is the same as for `wint_t`.
- *[Editor's note: See [\[cwctype.syn\]](#) and [\[cwchar.syn\]](#).]*
- Tom observed that the clash with `WEOF` is only a problem when the `WEOF` value is in the range of `wchar_t` values; e.g., when `WEOF` is `-1` and `wchar_t` is a signed type.
- Jens noted that the C standard requires that `wint_t` be able to hold all extended character values and that Hubert's concern is that C++ extends more flexibility to users in use of particular values.
- Tom indicated that he would work with Hubert to get an issue filed.
- Corentin stated that `std::char_traits<wchar_t>` also suffers from the lack of an available value for EOF in implementations like Microsoft's where both `wchar_t` and `wint_t` are 16-bit and used with UTF-16.
- *[Editor's note: Microsoft's implementation uses an unsigned 16-bit type for both `wchar_t` and `wint_t`, defines `WEOF` as `((wint_t)(0xFFFF))`, `WCHAR_MIN` as `0`, and `WCHAR_MAX` as `0xFFFF`. That leaves no values left for use as an EOF sentinel.]*
- Hubert expressed skepticism that such implementations are conforming.
- Jens recalled that changes were made to allow for use of UTF-16 with `wchar_t` at the core language level but that such allowances were not extended to the standard library.
- *[Editor's note: see [P2460 \(Relax requirements on `wchar\_t` to match existing practices\)](#).i]*
- Jens acknowledged that the distinction doesn't matter much since existing implementations are not going to be changed.
- Tom expressed a preference to fix `char_traits<char16_t>` as a technically breaking change.
- Jens requested that implementors be directly contacted for feedback.
- Hubert also encouraged Jens' request since a change would break use of `libc++` iostreams with `char16_t`.
- Jens acknowledged the potential break, but noted that the ability to use iostreams with `char16_t` might not be intentional.
- Jens presented `std::complex` as an example of a class template that has restrictions on which types are allowed as template type arguments.
- Alisdair stated that there are a number of class templates for which instantiations are only guaranteed to work with certain types.
- Tom asked for confirmation that `std::regex` is limited to instantiations with `char` and `wchar_t`.

- Alisdair confirmed that is his understanding.
- Corentin noted that fixing `std::regex` to properly support Unicode would require an ABI break.
- Tom turned discussion towards the issues concerning `std::codecvt`.
- Tom asked for confirmation of his expectation that everyone is in agreement that the `std::codecvt<charN_t, char8_t, std::mbstate_t>` specializations that should not have been added in the first place should be deprecated and removed.
- Victor replied with a thumbs up.
- Alisdair stated that the deprecated `std::codecvt<charN_t, char, std::mbstate_t>` specializations are only needed by implementors that want to support iostreams with the `charN_t` types.
- Tom agreed.
- Steve noted that those are specified with fixed UTF encodings.
- Jens stated that, as specified, those facets have the wrong semantics.
- Alisdair observed that the current semantics stand in the way of an implementor doing the right thing with iostreams of `charN_t` type.
- Jens agreed.
- Corentin claimed that there are two questions:
 - Whether we think `std::codecvt` is useful to users and whether we want to continue to support it in the standard.
 - How iostreams perform conversions.
- Corentin asserted that we don't have to rely on `std::codecvt` to implement conversions.
- Tom agreed, but noted that a new mechanism would presumably have to be applied only for the `charN_t` types so as not to interfere with iostreams of `char` and `wchar_t`.
- Steve stated that it isn't clear that the `std::codecvt` facets are doing what anyone wants.
- Tom observed that iostreams of `wchar_t` are pretty much only used on Windows and iostreams of `char` use a `std::codecvt` facet that does nothing by default.
- Alisdair requested that any proposed changes to the `std::codecvt` facets include discussion of how the virtual functions can be overridden to provide different behavior.
- Alisdair asked if any changes are required to P2873.
- Tom replied that he is leaning towards undeprecating those facets since the `char8_t` facets that were intended to replace them don't actually do so.
- Jens reiterated that the deprecated facets have the problem that they convert to the wrong encoding.
- Jens stated that, once removed, they could be reintroduced with new semantics.
- Tom replied that the facets have already been deprecated for two release cycles and that implementations diagnose them.
- Mark acknowledged the deprecation but pointed out that warnings are suppressed in system headers.

- Tom noted that warnings will have been generated for any explicit use of the deprecated specializations.
- Jens observed that the deprecation has only poisoned any existing `charN_t` `iostream` implementations and asserted that removing them is the clearest path forward.
- Jens claimed that removal sends a stonger message than deprecation for any existing uses.
- Corentin expressed support for removing them and then adding them again later if needed.
- Jens argued for focusing on cleanup in this release cycle rather than considering whether we want to add support for `charN_t` in `iostreams`.
- Tom turned discussion to the final issue; that the deprecated `std::codecvt<char16_t, char, std::mbstate_t>` facet doesn't satisfy the N:1 rule for `std::basic_filebuf`.
- Tom noted that the `wchar_t` specialization has this issue as well.
- Jens pointed out that it technically doesn't because the library does not permit UTF-16 for the wide encoding.
- *[Editor's note: see [\[character.seq.general\]p\(1,2\)](#).]*
- Jens asserted that we should not address this without a paper.
- Tom agreed.
- Hubert expressed his perception of where consensus is headed; that we are leaning towards a clean slate for a potential proposal to introduce `iostreams` of `charN_t`.
- Jens agreed.
- Tom interpreted that as an argument for Alisdair's paper going forward as is.
- Corentin stated that any paper that proposes `iostreams` for `charN_t` needs to explore use cases.
- Jens added that such a paper must also consider the current absence of `std::codecvt<char8_t, char, std::mbstate_t>` specializations.
- Tom agreed and argued that such specializations should not be added until there is a demonstrated need for them.
- Jens requested that Alisdair's paper clearly delineate what actions to take now vs what would be needed by a hypothetical proposal to introduce `iostreams` of `charN_t`.
- Alisdair stated he would like to update the rationale so as to better explain the situation to LEWG and then submit a revision for LWG for the post-Kona mailing.
- Steve suggested posting the revision to the SG16 mailing list for additional review.
- Tom discussed scheduling for the next SG16 meeting:
 - Tom announced that the next regularly scheduled SG16 meeting would conflict with the WG21 meeting in Kona and that the one after that conflicts with Thanksgiving in the US.
 - Tom suggested meeting on 2023-11-15 and 2023-12-06 and then pause until the new year.

- Jens objected that 2023-11-15 is too close to Kona post-meeting activities.
- Tom suggested meeting on 2023-12-06 and 2023-12-20.
- Victor stated he would not be available on 2023-12-20.
- Tom proposed that we meet 2023-12-06 and evaluate then whether to meet 2023-12-20 or suspend until the new year.
- *[Editor's note: in later [mailing list discussion](#) it was decided the group would meet again 2023-11-29 and 2023-12-13.]*

November 29th, 2023

Draft agenda:

- [P2980R0: A motivation, scope, and plan for a physical quantities and units library](#):
 - Support for a `fixed_string` type as referenced in the ["External dependencies" section](#).
 - Support for `std::format` and display of symbol names.
 - Support for `wchar_t`, `char8_t`, `char16_t`, and `char32_t`.

Attendees:

- Eddie Nolan
- Fraser Gordon
- Lauri Vasama
- Mateusz Pusz
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- A round of introductions was held for new attendee Lauri Vasama.
- [P2980R0: A motivation, scope, and plan for a physical quantities and units library](#):
 - Mateusz explained that the contents of this paper, as well as the contents of [P2981 \(Improving our safety with a physical quantities and units library\)](#) and [P2982 \(std::quantity as a numeric type\)](#) are being merged into a new paper following feedback during the Kona 2023 meeting.
 - Mateusz proceeded with presenting a draft version of the new paper.
- [P3045R0: Quantities and units library](#):
 - *[Editor's note: D3045R0 was the active paper under discussion at the telecon. The agenda and links used here reference P3045R0 since the links to the draft*

paper were ephemeral. The published document may differ from the reviewed draft revision.]

- Mateusz introduced the paper:
 - Formatting support is needed to present dimensions and units.
 - Unicode doesn't provide subscript and superscript characters for all Latin characters, so formatting necessarily differs from conventional notation in some cases.
 - The design currently specifies symbol names in terms of char and assumes a Unicode encoding.
 - A `fixed_string` type is required to enable a unit symbol to be passed as a template argument for the `named_unit` class template.
 - The library only requires a `fixed_string` type with read capabilities; mutation is not needed.
 - There are many implementations of a `fixed_string` type and re-inventing yet another one for this library is not desirable.
 - There are many design options for a `fixed_string` type including whether mutate and resize operations are supported or whether the type can be implemented with `std::string` and a fixed allocator.
 - `std::string_view` does not support mutation.
 - The conventional notation for SI units depends on characters that are not represented in ASCII or in the basic character set.
 - Some users will require ASCII-only output and there is no standard specification for ASCII-only symbol names.
 - Supporting both Unicode and non-Unicode formatting requires alternative symbols.
 - The `basic_symbol_text` class template allows for both a Unicode and ASCII-only representation to be provided.
- Tom mentioned that formatted output should be designed for roundtripping so that the output produced is amenable to scanning.
- Tom noted that a proposal for text parsing is making its way through the committee.
- [Editor's note: See [P1729 \(Text Parsing\)](#).]
- Mateusz agreed that roundtripping is important to support serialization to a text file and back.
- Tom suggested that, in lieu of a `fixed_string` type, string operations could be provided by layering `std::string_view` on top of a template parameter that provides contiguous storage.
- Mateusz agreed that `std::array` could be used.
- Tom acknowledged that `std::array` is a structural type and thus usable as a non-type template parameter.
- Eddie asked if `operator+` and other operators could be provided on top of `std::array`.
- Mateusz replied that he believed so.

- Lauri expressed concern that deduction guides might be problematic due to null terminators.
- Tom noted that the proposal assumes that a string literal is always passed as the template argument for symbol names.
- Lauri stated that the array approach won't work if there is special handling of string literals.
- Eddie suggested that a simple wrapper type with a `std::array` member and a suitable deduction guide could work.
- Tom suggested use of a UDL since they can only be used with a string literal.
- Mateusz replied that consideration should be given to this functionality being user facing.
- Steve stated that use of `std::array` instead of a more specific type could lead to ambiguities later.
- Lauri noted that a UDL would require another structural type.
- Tom agreed and acknowledged that use of a UDL would affect the interface and the user experience.
- Mateusz asserted that the parameter type should have associated text semantics and not just provide storage.
- Tom asked how important it is that the programmer be able to control whether symbols are formatted with Unicode or ASCII-only characters.
- Mateusz replied that there are some users that require ASCII-only output and that an inability to opt-out of a full Unicode mode would be a no-go.
- Mateusz stated that there isn't a similar concern for iostreams since a manipulator could be provided to control the mode.
- Tom stated this can remain an open question for now.
- Fraser suggested that the formatter could allow the programmer to specify an alternate unit symbol in the format specification itself.
- Victor noted that `std::print` works with iostreams, so iostream support could be provided indirectly.
- Victor asked if there are interactions with locale.
- Mateusz replied that the ability to provide locale support is limited by the standard not providing access to the Unicode CLDR database or similarly suitable locale support.
- Victor recommended reserving an 'L' option specifier in the format specification that would render the code ill-formed for now so as to allow extension later without an ABI break.
- Eddie noted that the standard already permits an implementation to choose between a Unicode and ASCII symbol for iostream formatting of `std::chrono::duration`.
- [Editor's note: see [\[time.duration.io\]p\(1,5\)](#):

Otherwise, if `Period::type` is `micro`, it is implementation-defined whether `units-suffix` is `"μs"` ("`|u00b5|u0073`") or `"us"`.

]

- Eddie opined that `char8_t` should probably be used for storage of the Unicode symbol name.
- Eddie asserted that the paper should substitute "basic character set" for "ASCII" throughout.
- Eddie noted that U+212B (ANGSTROM SIGN) has a tendency to get normalized to U+00C5 (LATIN CAPITAL LETTER A WITH RING ABOVE) or U+0041 (LATIN CAPITAL LETTER A) followed by U+030A (COMBINING RING ABOVE).
- Mateusz responded that, with regard to use of `char8_t`, that it was suggested to him to just use `char`.
- Tom replied that opinions differ on that.
- Steve asserted that the proposal should explicitly specify the code points to be used and should not rely on glyphs.
- Tom noted that the language specification has been updated to be explicit about code points, but that fewer such updates have been done for the library specification.
- Eddie asserted that normalization should be specified as well.
- Tom agreed and stated a preference for NFC.
- Eddie disagreed with the use of NFC since, per earlier discussion, U+212B (ANGSTROM SIGN) won't be preserved.
- Steve pointed out that, although the standard requires NFC for identifiers, it imposes no such requirement on string literals.
- After some back and forth it was pointed out that the precedent in the standard is that the code point used for `iostream` formatting of `std::chrono::duration` is U+00B5 (MICRO SIGN) rather than its normalized equivalent U+03BC (GREEK SMALL LETTER MU).
- Eddie opined that, given this precedent, we should not specify a normalization for units, and given multiple alternatives we should use code points corresponding to units, e.g. U+212B (ANGSTROM SIGN) rather than U+00C5 (LATIN CAPITAL LETTER A WITH RING ABOVE).
- Mateusz directed discussion to section 13.1.4.1 (`unit_symbol_formatting`) where various enumerations are defined to support encapsulating formatting in the `unit_symbol_formatting` class.
- Victor commented that the enumeration types in that section should have specified underlying types unless they are intended to be transient.
- Mateusz replied that the enumerations are only used at compile-time, but agreed that adding a fixed underlying type might still make sense.
- Mateusz explained that `space_before_unit_symbol` is provided as a customization point to control whether a space is inserted between a value and its unit symbol by default.
- Mateusz directed discussion to section 13.2.3.1 (`std::format Grammar`) and noted that the proposed grammar is similar to that for `std::chrono` with the addition of

options for text encoding, and controls for inserting a solidus or separator character.

- Victor observed that the `units-unit-modifier` seems odd since, as specified, it requires that if any of `units-text-encoding`, `units-unit-symbol-denominator`, and `units-unit-symbol-separator` is present, then they all must be.
- Victor asked whether each of those terms should appear separately in square brackets.
- Mateusz replied that the intent is that each term can optionally be present in an unordered sequence.
- Tom replied that specifying an order would avoid having to consider each term being present multiple times.
- Tom raised discussion of upcoming meeting plans:
 - Tom stated that the next meeting is scheduled for December 13th and that he would like to return to some LWG issues.
 - *[Editor's note: The December 13th meeting was canceled due to lack of sufficient progress on the LWG issues to warrant additional discussion.]*
 - Tom asked Mateusz if we can resume discussion of this paper on January 10th.
 - Mateusz replied that he is not available that week.
 - Tom asked if January 24th would work.
 - Mateusz replied affirmatively.
 - Mateusz requested a list of items to address or consider before the January 24th meeting so that he can work on them to try and get some implementation experience in the meantime.

January 10th, 2024

Draft agenda:

- [CWG 2843: Undated reference to Unicode makes C++ a moving target.](#)
- [P2626R0: charN_t incremental adoption: Casting pointers of UTF character types.](#)

Attendees:

- Alisdair Meredith
- Corentin Jabot
- Eddie Nolan
- Fraser Gordon
- Jens Maurer
- Mark de Wever
- Robin Leroy
- Steve Downey

- Tom Honermann
- Victor Zverovich

Meeting summary:

- Robin announced that the planned 2024-01-24 SG16 meeting overlaps with the UTC #178 meeting and that he will therefore be unable to attend.
- [CWG 2843: Undated reference to Unicode makes C++ a moving target](#):
 - Jens provided an introduction:
 - Undated references refer to the latest edition of such references.
 - The ISO prefers undated references.
 - WG21 negotiates the use of dated references with the ISO editors based on the fact that conscious effort is required to align wording and semantics with new editions.
 - The C++23 draft is still undergoing editorial changes in conjunction with the ISO.
 - The C++ standard used to have a normative reference to ISO/IEC 10646, but the reference was redirected to the Unicode Standard following additions that required features that are not specified in ISO/IEC 10646.
 - *[Editor's note: The change of normative reference was made via [P2736R2 \(Referencing The Unicode Standard\)](#).]*
 - ISO/IEC 10646 is not identical to the relevant portions of the Unicode standard.
 - The ISO has so far not complained about the C++ standard's use of the Unicode Standard despite the ISO generally preferring to refer to ISO standards.
 - The undated reference to the Unicode Standard is "live"; which means that, as soon as a new Unicode Standard is published, the reference automatically refers to that edition.
 - That implies that a conforming implementation of C++23 that uses Unicode 15 becomes non-conforming the moment that Unicode 16 is published.
 - Changes to Unicode algorithms could impose ABI breaks that create difficulties for implementors.
 - The proposed resolution is to require conformance with Unicode 15.
 - It has also been suggested that a minimum Unicode version be specified with an allowance for implementors to use a more recent version.
 - A reference to a particular Unicode version is beneficial even if an allowance is made for use of a later version.
 - Specifying both an undated and a dated reference would be weird.
 - Alisdair stated that issuing a DR has a similar effect to publication of a new edition of an undated reference, but differs in that the change happens under the auspices of WG21 rather than being imposed by an unaffiliated third party.

- Jens clarified that DRs are not ISO publications and that WG21 so far has not made use of the ISO procedures for issuing technical corrigenda for defects or amendments for enhancements.
- Robin objected to the notion of the Unicode Consortium being an unaffiliated third party and noted the formal liaison relationship with SC22.
- Steve opined that fixing the Unicode version to Unicode 15 is probably fine for C++23.
- Jens replied that C++23 is done with the exception of editorial changes being coordinated with the ISO and that any action taken for this CWG issue will target C++26.
- Steve reported that he tends to start observing use of new Unicode features within four to six months of the publication of a new Unicode version.
- Steve stated that new emoji are often the first new feature observed and that such text needs to be correctly processed.
- Steve asserted that waiting for the next C++ standard for support of a new Unicode version isn't viable.
- Steve agreed with the approach of specifying a minimum version with an allowance for implementors to upgrade at their discretion.
- Steve advised against implementors using different Unicode versions for different C++ standard conformance modes since doing so would invite ODR violations.
- Corentin expressed agreement with Steve's comments.
- Corentin asserted that implementors need to be able to keep up with the Unicode Standard at a faster pace than the C++ standard can.
- Corentin stated that it is likely not viable to support different Unicode versions for different C++ standard conformance modes.
- Corentin reported having tried to support multiple Unicode versions in a private project and that it didn't work well.
- Corentin noted that the Unicode Standard has a good history of maintaining backward compatibility and that changes made often address defects for which fixes are desirable.
- Corentin agreed that a dated reference in the C++ standard is useful to facilitate references to specific sections by number and name.
- Corentin opined that guidance for implementors to handle or avoid ABI issues in accordance with Unicode stability policies would be useful.
- Corentin suggested that a note that expresses that intent would be helpful.
- Corentin reported that Clang releases have stayed current with the most recent Unicode versions and will continue to do so.
- Alisdair expressed alignment with Jonathan's suggestion for the version of the Unicode standard to be implementation-defined for defect reporting purposes.
- Alisdair stated a preference for not requiring a minimum version so that implementors can provide options to enable backward compatibility with previous releases while remaining conforming.

- Eddie noted an advantage of an undated reference is that it avoids potential opposition to updating the normative reference to a newer version due to ABI concerns.
- Eddie explained that `std::format` already has the potential to lock in at compile-time features from the Unicode Standard that don't have a stability policy.
- *[Editor's note: Eddie later [clarified on the SG16 mailing list](#) some misconceptions regarding `constexpr` and `std::format`; implementors have flexibility to isolate ABI concerns using `if constexpr`.]*
- Eddie agreed that it would be a good idea to provide guidance to implementors regarding how to isolate ABI concerns.
- Robin recognized that, in the real world, modern compilers support C++11 despite C++11 no longer being an active ISO standard, and projects are still developed with it.
- Robin cautioned that, if the version of the Unicode standard is tied to the C++ standard version, then projects using an older C++ version could be using a 10+ year old Unicode version and that possibility is even more concerning than having to wait three years to use a newer version.
- Robin emphasized the Unicode Standard's stability guarantees.
- Robin noted that implementations of a Unicode algorithm impose a limit on what Unicode versions are compatible.
- Steve provided an example of such limitations; extended grapheme clusters (EGCs) were introduced after the initial Unicode release and the use of such features imposes a minimum version that is required.
- Robin noted in the chat that EGCs were introduced in Unicode 5.1 in April of 2008.
- Corentin expressed support for specifying a minimum Unicode version for portability reasons.
- Corentin stated that he is not concerned about ABI issues at this point and asserted they haven't been a practical issue for Unicode concerns so far.
- Mark replied that `libc++` does have an ABI issue that will need to be resolved; there is a table that needs to have an ABI tag applied to it.
- Mark expressed support for implementations being able to use newer Unicode versions because that is useful for users.
- Mark stated a preference for an implementation-defined version rather than one which must be adhered to.
- Alisdair indicated that he would be content to have market pressures determine compatibility.
- Alisdair stated a desire for an allowance for a conforming implementation to support use of an older Unicode version for compatibility with prior C++ standard versions.
- Alisdair stated in chat: "Conversely, I would not object to a "recommended practice" to set the floor, rather than making it normative".
- Eddie asked if there is an ABI impact from `std::format` width estimation changes.

- Mark replied that the width estimation in libc++ is constexpr as an implementation detail.
- Tom expressed a belief that width estimation has to be performed with run-time field values.
- Corentin acknowledged that the C++ standard may need to refer to a minimum Unicode Standard version just to be able to refer to certain features.
- Corentin asserted that there are ways that implementors can hide things behind ABI and that this includes use in constexpr context.
- Jens agreed with Alisdair that the Unicode version actually used in a particular language mode should be implementation-defined.
- Jens disagreed about not specifying a definite minimum version.
- Jens explained that core language features like named universal characters (`\N{...}`) require a minimum Unicode version in order to write portable code.
- Jens asserted that features that can't be reliably used across implementations should be removed.
- Jens observed that a consistent version of the Unicode Standard is required in order for the C++ standard to be consistent.
- Jens opined that the C++ standard should not reference different Unicode versions for the core language and the standard library.
- Tom asked if it might make sense for the minimum Unicode Standard version required for implementations to conform to the C++ standard to be different from the normative dated reference.
- Jens replied negatively.
- Jens stated that the formal text needs to provide the right guarantees even if implementors all do what we consider to be the right thing; the formal text must be sufficient to write portable programs.
- Jens noted that the ISO will not permit the introduction of an alias for a normative reference.
- Jens expressed uncertainty where a Unicode version conformance requirement should be specified, but stated that is likely a solvable problem.
- Jens observed that identifiers have a forward compatibility guarantee thanks to the Unicode Standard stability policies for XID start and continue properties.
- Steve reported that his organization builds their internal toolchain using system supplied libraries and noted this could produce a non-conforming implementation due to building with older Unicode libraries.
- Steve indicated he is ok with that result though.
- Steve noted that the Unicode Standard is a coherent specification and that mixing parts from different versions of it can produce non-sensical results.
- Steve described "ABI problems" as shorthand for lots of different problems, some of which, like virtual function table layout differences, are catastrophic while other cases, like fast math enabled vs disabled, are not.
- Alisdair stated that he has been persuaded by Jens' arguments that a dated reference to the Unicode Standard in the C++ standard with the actual version

being implementation-defined is a good direction.

- Alisdair opined that it is still important for implementors to be able to provide backwards compatibility and that he would prefer normative guidance for use of the normative dated reference to be the minimal version supported by an implementation.
- Gordon explained that the ISO prefers undated references because ISO standard editions effectively disappear when superseded and asked for clarification that the Unicode Consortium handles this differently.
- Robin confirmed that release of a new Unicode Standard does not obviate the preceding ones and provided a link to <https://www.unicode.org/versions> in the chat.
- Robin asked for Jens to confirm that, with regard to named universal characters, whether the concern is in regard to upgrading compilers.
- Jens explained that implementations might want to issue a portability warning for use of a name that was added in a later Unicode Standard than the dated version from the C++ Standard.
- Jens reported that he wants to be able to rely on all character names from, e.g., Unicode 15, being available for use across all C++ implementations.
- Robin asked if the same concern applies to identifiers.
- Jens confirmed that it does.
- Robin explained that, as long as the C++ standard specifies a minimum version and that implementations are permitted to use a newer version, then he is content; he would not be content with the C++ standard specifying a maximum version though.
- Steve noted that implementations are free to accept ill-formed code as long as a diagnostic is issued.
- Alisdair asked whether a feature test macro with predictable values can be specified.
- Jens noted that the C++ standard currently provides the `__STDC_ISO_10646__` macro with a date value.
- Corentin replied that the existing macro can't be relied on at compile-time because it is shared between the core language and the standard library.
- Robin reported that the Unicode Standard does not have a stability policy for the format of the Unicode version but stated that such a policy could be proposed.
- Jens replied that the year and month of the release date suffices assuming the Unicode Consortium doesn't start shipping new releases at a rate higher than once a month.
- Tom summarized his perception of the emerging consensus:
 - The C++ standard should have a single dated reference to the Unicode Standard for consistency purposes.
 - A minimum Unicode version should be specified as normative guidance or as a mandatory requirement.
 - The actual Unicode version in use by an implementation should be implementation-defined and allowed to be newer than the minimum version.

- The Unicode version in use by an implementation may differ for the core language vs the standard library; separate feature test macros may be required to identify the implementation-defined version.
- Jens noted that the minimum version may be increased in future C++ standards to accommodate references to features introduced in newer versions.
- Tom observed that some effort will be required to identify the minimum Unicode version required for the C++ standard.
- Suggestions were made to specify Unicode 15 as the minimum version.
- **Poll 1: Recommend having a dated reference to Unicode in the "Normative references" and add permission to implement an implementation-defined version.**
 - **Attendees: 10**
 - **No objection to unanimous consent.**
- **Poll 2: The standard shall specify a mandatory minimum Unicode version.**
 - **Attendees: 10**
 - **SF F N A SA**
3 5 1 1 0
 - **Consensus in favor**
 - **A: I would prefer to allow implementations to use older Unicode versions and still be considered conforming; implementations will do so regardless.**
- Steve summarized the consensus: we recommend having a dated reference to the Unicode Standard in the "Normative references" section, a minimum version requirement, and an allowance for implementors to use an implementation-defined later version.
- Jens stated that he will update the proposed resolution for the CWG issue to reflect the SG16 consensus.
- [P2626R0: charN_t incremental adoption: Casting pointers of UTF character types:](#)
 - Tom thanked Corentin for agreeing to defer discussion of this paper.
- Tom reported that the next meeting will be in two weeks and will continue review of Mateusz' paper as well as additional followup on the CWG issue.

January 24th, 2024

Draft agenda:

- [P3045R0: Quantities and units library.](#)
- [CWG 2843: Undated reference to Unicode makes C++ a moving target.](#)

Attendees:

- Billy Baker
- Corentin Jabot
- Eddie Nolan
- Elias Kosunen
- Fraser Gordon
- Lauri Vasama
- Mark de Wever
- Mateusz Pusz
- Nathan Owen
- Jens Maurer
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- [P3045R0: Quantities and units library](#):
 - Mateusz provided an introduction:
 - There is a need for some unit types to have both a basic unit symbol and one that includes characters that are not in the basic literal character set.
 - The proposed design allows specifying multiple symbols.
 - We need to decide how these different symbols are specified.
 - Tom asked what character types need to be supported.
 - Corentin recalled an LWG issue concerning the symbol used to print `std::chrono::duration` values with a microseconds period and that the issue was resolved in favor of allowing the implementation to choose between two symbols.
 - *[Editor's note: See [LWG #3094 \(§\[time.duration.io\]p4 makes surprising claims about encoding\)](#) and the current wording in [\[time.duration.io\]p\(1.5\)](#).]*
 - Corentin suggested that precedent could be followed here.
 - Corentin opined that there is not much motivation for `wchar_t`, `char16_t`, and `char32_t`.
 - Mateusz responded that there was only one such case to be addressed for the chrono library but there are many such cases for the units library.
 - Mateusz added that there is a desire to allow programmers to restrict formatting to basic characters so as to avoid non-basic characters being written in some cases.
 - Mateusz acknowledged that removing the need for multiple symbols would simplify the design.
 - Victor agreed with Corentin and argued for a design that is simple and prioritizes Unicode.
 - Victor stated it should not be necessary to spell out symbols for all five encodings.
 - Victor concurred that the `std::chrono::duration` example is a good model to follow.

- Tom expressed skepticism regarding an implementation-defined approach since the units library is designed to be user extensible.
- Tom expressed a preference to specify a design that will work for user code.
- Corentin replied that the symbols for the unit types defined by the standard library could be implementation-defined.
- Corentin observed that passing arbitrary string literals as template arguments could cause compatibility issues if a program includes translation units built with different choices of the ordinary literal encoding.
- Corentin shared <https://godbolt.org/z/8frTvfvoE> as an example that demonstrates the concern.
- *[Editor's note: The concern is that a string literal like "μ" might be differently encoded such that the specialization `prefixed_unit<{"μ", "u"}, ...>` might not coincide across translation units.]*
- Corentin expressed uncertainty regarding catering to programmers that want to avoid seeing non-ASCII characters.
- Mateusz replied that the concern isn't just for reading the formatted output but that people need to be able to write the characters as well.
- Mateusz reported that there is no standard for ASCII-only symbol names.
- Steve agreed that the choice of ordinary literal encoding can create portability problems.
- Steve advised caution regarding potentially requiring the ordinary literal encoding to be able to accommodate characters not in the basic literal encoding.
- Elias observed that specifying the symbols as implementation-defined would cause problems for exchange of text.
- Steve noted that C++23 requires a conforming implementation to support UTF-8.
- Tom agreed, but noted that the UTF-8 requirement is for the encoding of source files and that the ordinary literal encoding need not support UTF-8.
- Steve observed that the proposed design would therefore be unimplementable for some implementors.
- Mark opined that it would be useful to specify alternate symbols for implementations to use.
- Corentin asserted that ordinary character and string literals can't be used as template arguments due to the possibility of inconsistent ordinary literal encoding.
- Mateusz pondered whether a Unicode encoding should be used for all the symbols.
- Corentin replied that he thinks that is necessary to avoid compatibility problems.
- Steve observed that the compiler can't correct for such incompatibilities because this is effectively a linkage concern.
- Elias asked if there is a compelling reason for the symbol names to be provided as template arguments.
- Mark replied that the motivation is to enable a succinct programming style as opposed to specializing a trait.

- Victor opined that the symbol is data and should not be specified as part of the type.
- Victor argued that moving the symbol out of the type system would make the design less fragile.
- Victor stated that macros can be used to provide a succinct programming style.
- Steve raised a concern that making data part of the type can lead to accidental ABI freezes where, for example, misspellings can't be fixed.
- Steve noted that such a design limits future extension possibilities as well.
- Tom asked Mateusz how moving the symbols out of template arguments would impact the design.
- Mateusz replied that users appreciate the terseness the current design allows and stated that exposing macros as part of a standard interface would not be desired.
- Mateusz acknowledged such a change would be possible though.
- Elias cautioned that we don't have an alternative design in front of us to consider and that makes it difficult to evaluate relative benefits.
- Mateusz stated that strong types are important to the design.
- Mateusz suggested A CRTP-based design could work.
- Victor stated that it seems problematic to have the symbol text be part of the identity of the type.
- Victor suggested that tag types would be more appropriate.
- Mateusz reported that he ran into difficulties when considering tag types but that he needs to explore some more.
- Mateusz stated that use of tag types would change the interface considerably.
- Steve returned discussion to support of multiple encodings and asserted that use of transliteration should be avoided since it can produce surprises like "Ω" (U+03A9 GREEK CAPITAL LETTER OMEGA) getting converted to "O".
- Tom summarized his impression of where the discussion has been leading:
 - The proposal authors should explore alternatives to passing symbols as template arguments.
 - There does appear to be a need to specify symbol alternatives for different encodings.
 - A method of specifying a symbol alternative in a UTF form and another as an ordinary string literal should suffice to support all five encodings.
- Victor reiterated that exploration of alternative designs should include the option of implementation-defined symbol selection.
- Tom replied that there is still a need to specify symbol selection for user-defined units.
- Corentin agreed that there appears to be consensus for a fallback symbol to be used when the preferred symbol is not representable.
- Corentin expressed uncertainty regarding consensus for a user opt-in to use of a fallback symbol.
- Mateusz directed discussion toward use of '_' to indicate a subscripted character in cases where Unicode lacks a corresponding character.

- Steve stated that subscripted characters in Unicode exist solely for compatibility with legacy character sets and that subscripting and superscripting are considered markup.
- Corentin opined that if subscripting and superscripting can't be done uniformly everywhere, then it should not be done anywhere.
- Corentin suggested consulting with Robin.
- Corentin wondered whether the ISO standards on units suggest a solution.
- Jens stated that he doesn't think there is a portable way to represent physics symbols in ordinary string literals.
- Jens suggested that it should be possible to allow a user to insert markup for support of subscripting and superscripting.
- Jens questioned whether support for non-ASCII characters should be provided at all since plain text can't represent the desired formatting.
- Mateusz replied that others have provided similar feedback such as the ability to produce LaTeX.
- Mateusz stated that he doesn't know how to do that with `std::format` or `std::print` though.
- Corentin agreed with Jens that users will want more capabilities and that these symbols are intended for display in a terminal.
- Steve suggested that, since the library is intended to support user-defined units, perhaps the unit symbols defined by the standard library should be restricted to the basic literal character set and programmers can use whatever characters from the actual ordinary literal encoding that they like for their own unit types.
- Steve commented that the symbol is significant in the type system.
- Jens agreed that it is and that units need to be preserved such that `2*speed_of_light == speed_of_light`.
- Victor agreed with Jens that we shouldn't put too much effort into pretty formatting since users can perform their own formatting.
- Victor asserted that the main purpose of the library is to provide the unit primitives as opposed to nicely formatted output.
- Mateusz asked if `std::format` could potentially take a tag type to differentiate behavior.
- Victor replied that the way to differentiate behavior would be to write separate formatters.
- Jens noted that the way to opt-in to such differentiated behavior is to wrap types accordingly.
- Jens suggested updating the narrative of the paper to demonstrate how to produce nicely formatted output for these types.
- Jens indicated that it would be nice to be able to specify custom formatting with a terse syntax.
- Mateusz expressed uncertainty regarding how, for example, a `std::vector` of these types could be formatted in a custom way.

- Jens acknowledged uncertainty regarding whether the `std::vector` formatters could handle that.
- Jens observed that a `std::vector` wrapper could presumably apply a corresponding wrapper to its elements.
- Jens suggested that an inability to do so might imply a deficiency in `std::format` that might be worth addressing and stated that an HTML formatter shouldn't require reinventing `std::format`.
- Eddie opined that, even if formatted symbols are only used for debug-like scenarios, Unicode support is useful and should be a goal.
- Mateusz reported that none of the units libraries that he is aware of provide such extensive formatting capabilities.
- Jens opined that such capabilities are not needed for the standard either but that it would be useful to illustrate what a solution might look like.
- Steve asked for additional topics that would benefit from discussion.
- Mateusz asked for preferences regarding the return type of `unit_symbol()`.
- No opinions were offered.
- Mateusz stated that adding additional iostream manipulators is probably not desirable and recalled that previous discussion settled on just providing `std::format` support.
- Tom asked Victor if there is an SG16 concern regarding section 13.4.1, "Controlling width, fill, and alignment".
- Victor replied that the behavior should be consistent with other formatters and that any reason to deviate should be discussed.
- Jens asked for confirmation that nested formatting works with ranges.
- Mark and Victor both confirmed.
- Mateusz stated that the proposal uses nested `{}` braces for formatting of subentities.
- Victor expressed opposition to use of `{}` for nesting because it closes off syntax space that could be used for other extensions.
- Victor noted that there are other delimiters that can be used.
- Mateusz stated that the parse context isn't copyable, so there isn't a portable way to handle nesting.
- Victor replied that implementation is straight forward using implementation internals.
- Jens noted that, for the purposes of standardization, it doesn't matter if the subentity selection is portably implementable using existing implementations.
- Corentin stated that the proposed approach doesn't support localization.
- Tom noted that message formatting capabilities would be required for that.
- [CWG 2843: Undated reference to Unicode makes C++ a moving target](#):
 - Tom apologized for the lack of time for further review of this issue.
- Tom announced that the next meeting will be 2024-02-07.

February 7th, 2024

Draft agenda:

- Updates from the Unicode liaison from the UTC #178 meeting.
- [CWG 2843: Undated reference to Unicode makes C++ a moving target.](#)
- [P2845R6: Formatting of std::filesystem::path.](#)
- [P3070R0: Formatting enums.](#)

Attendees:

- Eddie Nolan
- Jens Maurer
- Mark de Wever
- Nathan Owen
- Peter Bindels
- Robin Leroy
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- Updates from the Unicode liaison from the UTC #178 meeting:
 - Robin shared the following updates:
 - Draft meeting minutes are available at <https://www.unicode.org/L2/L2024/24006.htm#178-0>.
 - Character assignments may now be specified on a provisional basis to facilitate early feedback and development; this is particularly useful for font development.
 - ICU will not expose characters in alpha or beta status.
 - Product releases should not include support for provisional character assignments.
 - Alpha review for Unicode 16.0 started yesterday; background material is available at <https://www.unicode.org/review/pri497/pri497-background.html>.
 - Unicode 16.0 will specify new normalization behavior that might invalidate optimization techniques used by some implementations.
 - A conformance testsuite is available that exercises the new normalization behavior.
 - There was a minor update to [UTS #55](#) for case insensitive identifiers.
 - *[Editor's note: See the changes to [section 3.1.1, "Normalization and Case", in the 2024-01-03 proposed update of UTS #55](#).]*
 - Fraser Gordon was nominated and confirmed to chair the Terminal Text Working Group.

- The ICU technical committee has created a new Inflection Working Group.
- Tom noted that the new Inflection WG would presumably be relevant to the Message Formatting Working Group.
- Robin agreed.
- [CWG 2843: Undated reference to Unicode makes C++ a moving target](#):
 - Tom explained that, following decisions made during the [2024-01-10 SG16 meeting](#), we now need to select a Unicode version for the standard to refer to.
 - Steve proposed using the version that was current when designs were being evaluated and wording drafted.
 - Steve observed that doing otherwise might result in references that don't exist in the normatively referenced version or that behavior or features might have changed.
 - Steve stated that, for most features adopted during the C++23 development cycle, that would probably be Unicode 15.
 - Robin reported that Unicode 15.1.0 has material differences due to changes inspired by SG16 and the [UTS #55 \(Unicode Source Code Handling\)](#) effort that impacted the `XID_start` and `XID_continue` properties.
 - Robin noted that Unicode 15.1.0 also has changes for EGC segmentation for Indic scripts, shared a link to the [Sample Grapheme Clusters table in UAX #29 \(Unicode Text Segmentation\)](#), and referenced the Devanagari *kshi* example (क्षि).
 - Robin observed that the current undated reference currently resolves to Unicode 15.1.0.
 - Eddie indicated a desire to ensure that implementors can defer to ICU for normalization and be free to choose which ICU version they use.
 - Eddie reported that, following discussion with Zach, he was convinced to use the latest Unicode version which is currently 15.1.0.
 - Eddie stated that implementors should be able to use different Unicode versions for the core language and the standard library.
 - Steve pointed out that there are multiple options for an ICU version to defer to; if they choose to defer to one supplied by the platform, then they could get stuck with an old version.
 - Eddie replied that is motivation for implementors not to defer to a platform supplied version or for granting permission for use of an older version.
 - Steve noted that Linux distributors like RedHat support the installation of new compiler versions on older OS releases.
 - Steve reported having encountered issues due to use of old versions of some platform supplied libraries.
 - Steve stated that we need to allow time for implementors to adapt to changes to the normatively referenced version.
 - Tom asked Jens if he will want EWG to review the choice of normative Unicode version reference.
 - Jens replied that this issue has wide visibility and that the related GitHub issue is tagged for EWG and LWG as well as SG16.

- Jens added that LWG can forward any concerns they have to LEWG.
- Jens stated that CWG will only be involved to vet the actual wording changes and the guarantees regarding availability of character names as needed for the core language.
- Mark commented that, if libc++ were to start relying on ICU, that such reliance would likely be expected to be satisfied by a distribution provided by the target platform.
- Mark stated that use of ICU would likely be determined on a per-feature basis.
- Eddie argued that such expectations suggest standardizing the lowest version that still covers everything in the standard.
- Jens noted that, at present, that lowest version is the most recent Unicode version due to the undated reference in C++23.
- Jens expressed being comfortable with specifying Unicode 15.0.
- Jens stated an expectation that implementors will likely honor the resolution of this CWG issue for C++23 if it is approved as a DR.
- Jens suggested that some implementors might choose to warn on use of features from newer Unicode versions.
- Robin reported that it is possible to subdivide ICU to include only necessary components.
- Robin added that it shouldn't be assumed that an implementor needs to rely on a version distributed with the platform.
- Steve stated that ICU has support for symbol versioning and that this would allow an implementor to distribute their own version such that it will not conflict with other versions.
- Jens suggested that future paper authors be encouraged to comment on whether implementations should or should not rely on ICU for particular features and the potential to get stuck with a dependency on an older version.
- Jens advocated for collecting opinions from implementors.
- Robin asserted that specifying Unicode 15.1.0 will help to position implementors for future upgrades.
- Steve claimed it would be useful to give implementors advanced notice.
- Tom asked if anyone knows what ICU version Microsoft provides and whether any implementations defer to it today.
- Mark reported that Microsoft relies on the platform ICU version for timezone data, but not for `std::format()` related features.
- *[Editor's note: [Microsoft's ICU documentation](#) does not report an ICU version, but does indicate that only C APIs are exposed due to the lack of a stable ABI for C++.]*
- Steve asserted that we should not use a normative reference for a version prior to Unicode 14.0.
- Steve stated that wording review would be necessary to determine if Unicode 13.0 matches the required features and intended semantics for recently adopted papers.

- Eddie asked whether SG16 would be ok if, for [P2729 \(Unicode in the Library, Part 2: Normalization\)](#), implementors wanted to use the version of ICU provided by the platform.
- Tom replied that he thinks implementors have options available to them to meet requirements; they might not love any of the options, but they do exist.
- Steve asserted that we need to make it clear to implementors that they must use consistent implementations of the Unicode algorithms.
- Eddie agreed and disclosed that there is also an unpublished proposal for segmentation.
- Eddie reported that there is a long history of security vulnerabilities that occurred due to use of parsers that interpreted the same text inconsistently.
- Robin informed the group that ICU does not provide default tailoring support.
- Steve responded that the base tailoring algorithms are not terribly difficult to implement but that some data is required.
- **Poll 1: Recommend specifying Unicode 15.1.0 as the minimum Unicode version for C++23 (as a DR) and C++26.**
 - **Attendees: 9**
 - **SF F N A SA**
3 5 1 0 0
 - **Consensus in favor**
- Tom stated that, with regard to Unicode versions being consistent across the core language implementation and the standard library, that it doesn't seem feasible to not allow divergence.
- Steve commented that problems caused by a mismatch are unlikely to be worse than processing text from other sources.
- Eddie noted that it is common to use Clang with libstdc++ and libc++ and that EDG does not provide a standard library implementation.
- Mark reported that different people tend to work on the compiler and the standard library and that the versions of each can be mixed; requiring a consistent Unicode version would be very hard.
- Steve took a devil's advocate role and suggested that, perhaps such cases are just not conforming.
- Steve stated that it is not required for all deployments to be conforming; non-conforming is not the same as useless.
- Steve opined that the standard should still acknowledge the possibility of mismatched versions.
- Robin noted that the standard library does not currently require a normative reference to Unicode for any of its features at the moment.
- Jens expressed a preference for treating the C++ standard as a unit and only normatively require a single Unicode version with allowances for use of a later version.
- Tom agreed, but stated a desire to provide programmers the ability to query the version in use.

- Jens replied that preprocessor behavior is impacted by Unicode version and that it is therefore unclear how useful a feature test macro would be.
- Steve suggested that we might be getting ahead of ourselves in asking what we would use a feature test macro for.
- Jens posited that a library version query utility of some kind might be more useful than a feature test macro.
- Jens stated that certain features can just be avoided for core language.
- Jens opined that it could be useful to write a `#error` directive based on Unicode version.
- Tom concluded that we should avoid specifying a feature test macro and an explicit allowance for the core language and standard library to use different Unicode versions until more need is identified.
- Tom stated that he will forward the CWG issue with the above poll.
- [P2845R6: Formatting of `std::filesystem::path`:](#)
 - Victor introduced the recent changes.
 - The `path-format-spec` now supports a `g` option to enable formatting a path as a generic path.
 - Discussion in chat confirmed that `/` is used as the path separator when formatting a generic path and that the native path separator is used otherwise.
 - **Poll 2: Forward P2845R6 to LEWG.**
 - **Attendees: 9**
 - **No objection to unanimous consent.**
- [P3070R0: Formatting enums:](#)
 - Victor explained the motivation for the new feature:
 - This allows defining a `format_as()` function rather than writing a `std::formatter` specialization.
 - This approach is simpler and more efficient at run-time.
 - Jens asked if an alternate type presentation can be requested in the format specifier.
 - Victor replied that a `std::formatter` specialization is required to do that.
 - Tom asked if the format specifier has to be `{}`.
 - Victor replied that it doesn't, that the format specifier is parsed according to the mapped type; the type returned by the `format_as()` customization point.
 - Jens asked if `format_as` is an existing customization point.
 - Victor replied that it is not; it is new with this proposal.
 - Eddie observed that the proposed functionality seems useful for many types, but that the proposal is restricted to enumeration types.
 - Victor responded that it is extensible to other types, but is limited to enumeration types for now due to lack of experience with other types.
 - Mark asked how field widths are handled.
 - Victor replied that they are handled the same as for the mapped type.
 - Eddie asked for confirmation that, as proposed, an attempt to use this feature for a type other than an enumeration type will fail.

- Victor confirmed the intent, but noted that the proposed wording is currently missing a constraint.
- Jens asked if the mapping is applied recursively and what happens if `as_format()` returns another enumeration type.
- Victor replied that it should work, but that he needs to check and then update the paper accordingly.
- Peter observed that this approach doesn't solve the problem of wanting to format the name of an enumerator.
- Victor agreed that mapping an enumerator value to a name still has to be explicitly written but that reflection would make that easy.
- **Poll 3: Forward P3070R0 to LEWG.**
 - **Attendees: 9**
 - **No objection to unanimous consent.**
- Tom announced that the next meeting will be on 2024-07-21 and that the agenda is TBD.

February 21st, 2024

Draft agenda:

- [CWG 2843: Undated reference to Unicode makes C++ a moving target.](#)
 - Identify updates needed for UAX #31 changes in Unicode 15.1.0.
- [LWG 4043: "ASCII" is not a registered character encoding.](#)
- [LWG 4044: Confusing requirements for `std::print` on POSIX platforms.](#)

Attendees:

- Eddie Nolan
- Fraser Gordon
- Jens Maurer
- Nathan Owens
- Peter Bindels
- Robin Leroy
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- [CWG 2843: Undated reference to Unicode makes C++ a moving target:](#)
 - Tom provided a brief introduction:

- Unicode 15.1.0 introduced changes to [default identifier syntax](#) to allow U+200C (ZERO WIDTH NON-JOINER) and U+200D (ZERO WIDTH JOINER) in identifiers.
 - We can choose to accept these changes or to adopt a profile that retains the prior behavior.
 - Regardless, the removal of [UAX31-R1a](#) necessitates an update to [\[uaxid.def.rfmt\]](#) in [Annex E](#).
- Steve stated that it makes the most sense to defer to Unicode for valid identifier syntax and for individual projects to decide what constitutes a reasonable identifier.
- Steve asserted that following Unicode guidance should not be an on-going discussion for WG21.
- Jens reminded the group that we decided to defer to Unicode explicitly so that we would not have to decide what is a valid identifier.
- Robin explained that this topic is on the agenda because there was a change to UAX #31 and Annex E now has dangling-ish references.
- Robin reported that the change made to default identifiers was a simplification and that [UTS #55 \(Unicode Source Code Handling\)](#) gives general guidance for identifiers.
- Robin noted that the provided guidance suggests adopting a profile from [UAX #31 section 7.1](#) to allow additional characters that are not included in default identifiers.
- Robin stated that some implementations already allow those characters and that formally adding them to C++ should probably be pursued by a separate paper.
- Tom noted that those additional characters are for some mathematics symbols.
- Steve agreed that is something to consider, but is unrelated to the current issue.
- Tom asked if anyone had an argument to offer for why we should not accept the UAX #31 updates.
- No such arguments were offered.
- Tom asked for a volunteer to update annex E.
- Steve volunteered.
- Robin expressed interest in collaborating on a paper to adopt the [Mathematical Compatibility Notation Profile](#).
- Tom requested that Steve send updated wording to Jens to be included in the proposed resolution.
- Jens stated that the proposed resolution will require approval from EWG due to the minimum version requirement.
- [LWG 4043: "ASCII" is not a registered character encoding](#):
 - Tom provided a brief introduction:
 - Users expect "ASCII" to be a recognized encoding name, existing converters recognize it, the proposed resolution is that it be recognized as an alias of "US-ASCII".
 - Fraser asked if it is known why "ASCII" isn't already an alias for "US-ASCII" in the [IANA character set registry](#).

- Tom guessed that it is due to historic confusion regarding "extended ASCII" character sets.
- Tom shared a link to the IANA character set reference and quoted the first paragraph in chat.

These are the official names for character sets that may be used in the Internet and may be referred to in Internet documentation. These names are expressed in ANSI_X3.4-1968 which is commonly called US-ASCII or simply ASCII. The character set most commonly use in the Internet and used especially in protocol standards is US-ASCII, this is strongly encouraged. The use of the name US-ASCII is also encouraged.

- Steve noted that the IANA registry includes a "csASCII" alias.
- Fraser opined that this sounds like a historic issue.
- Steve recalled that some special handling for "cs" prefixed names was adopted.
- Tom replied that the `std::text_encoding::id` enumerators use the "cs" prefixed aliases with the "cs" prefix removed.
- Tom asked if anyone is opposed to adding the proposed "ASCII" alias.
- Jens noted that implementations already have latitude to add additional names.
- Jens agreed we should add this particular alias, but not as a precedent for adding additional aliases later.
- Peter stated that ASCII is deserving of special consideration and is recognized around the world.
- Victor opined that the motivation in the LWG issue is a little weak, but that he isn't opposed.
- Tom reported that `iconv()` and ICU will already recognize it and opined that users will expect it to be recognized.
- Steve noted that implementors don't need our approval to add this alias.
- **Poll 1: Approve the addition of "ASCII" as an alias for the US-ASCII IANA encoding.**
 - **Attendees: 9**
 - **No objection to unanimous consent.**
- [LWG 4044: Confusing requirements for `std::print` on POSIX platforms:](#)
 - Victor introduced the issue:
 - Jonathan Wakely implemented support for `std::print()` in `libstdc++` and encountered a significant performance issue due to how he interpreted the standard wording.
 - When discussing `std::print()` in SG16, we didn't consider POSIX streams as a "Native Unicode API".
 - Private correspondence with Jonathan clarified the intent and resolved the performance issues.
 - Victor guided discussion through the proposed wording.
 - Victor highlighted the removal of the POSIX and `isatty()` related wording as the important change.

- Victor suggested that the moved text that encourages implementations to diagnose invalid code units be removed.
- Eddie agreed with striking the wording regarding diagnosing invalid code units.
- Eddie noted that checking for ill-formed code unit sequences imposes overhead.
- Steve asserted that `isatty()` is fragile and that its use complicates debugging since it leads to file redirection changing program behavior.
- Eddie asked Steve for clarification.
- Steve replied that `isatty()` is fragile because it is easy to cause `isatty()` to return false when the output is still going to the terminal.
- *[Editor's note: Compare the behavior of `ls` vs `ls | cat` on Linux for example.]*
- Eddie opined that tools should check the `NO_COLOR` environment variable.
- Steve insisted that `isatty()` is too low level for what `std::print()` is intended to do.
- Tom expressed support for dropping the wording regarding diagnosing invalid code units.
- Tom asked if the wording should state something else regarding the behavior when invalid code unit sequences are present but concluded that likely falls under implementation-defined behavior related to use of the native Unicode API.
- Jens asked if the Windows checks for code directed to a console have similar overhead concerns as calls to `isatty()` on POSIX systems.
- Victor replied affirmatively but noted that there is no known alternative at present.
- Victor stated that the check could become a no-op in the future if it becomes possible to check for use of a Unicode code page instead.
- Victor summarized that we can either do the wrong thing quickly or the right thing slowly.
- Jens expressed agreement for striking the wording regarding diagnosing invalid code unit sequences.
- Peter opined that the wording appears to be written from a Windows point of view and seems quite strange from a POSIX perspective.
- Peter suggested that the wording could discuss Windows specifically.
- Jens replied that Windows is specifically addressed in a note.
- Tom acknowledged that Peter has a valid point; the "native Unicode API" is only needed when writing directly to the stream is insufficient to produce the right result.
- Eddie advised caution regarding discounting the possibility that writing directly to the stream could produce the right result on Windows.
- Tom noted that Microsoft does ship versions of Windows that only support UTF-8 as the active code page and offered HoloLens as an example.
- Steve asked if implementors need this guidance.
- Victor replied that they do and that it took considerable exploration to determine exactly which functions were needed to achieve the right results.
- Jens commented that this is one of those rare places in the standard where we try to tell implementors what to do rather than just specifying the required behavior.

- Jens stated that the wording needs to be sufficient to guide implementors to the right result.
- **Poll 2: Approve the LWG 4044 proposed resolution with the wording about diagnosing invalid code units removed.**
 - **Attendees: 9**
 - **No objection to unanimous consent.**
- Tom announced that the next meeting will be on 2024-03-13; the week before the Tokyo meeting.
- Tom requested suggestions for any papers or issues that need SG16 review prior to Tokyo.